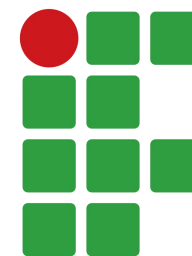


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Firebase

- Firebase é uma plataforma do Google que oferece serviços de backend e ferramentas para desenvolvimento de aplicativos.
- Permite adicionar recursos ao aplicativo sem a necessidade de desenvolver e manter toda a infraestrutura de servidor.
- Oferece serviços como:
 - Banco de dados
 - Autenticação de usuários
 - Armazenamento de arquivos
 - Notificações
 - Monitoramento e análise



Firebase

- O Android Studio possui ferramentas para facilitar a integração de projetos Android com o Firebase.
- A integração pode ser acessada em:
 - Tools → Firebase
- O Firebase Assistant auxilia na:
 - Conexão do aplicativo com um projeto Firebase;
 - Configuração das dependências necessárias;
 - Adição dos serviços do Firebase ao projeto.
 - Após a integração, os serviços podem ser utilizados diretamente no código do aplicativo.



Firestore Realtime Database

- Serviço de banco de dados NoSQL hospedado na nuvem.
- Permite armazenar e sincronizar dados em tempo real entre os dispositivos conectados.
- As alterações realizadas no banco podem ser propagadas automaticamente para os clientes.
- Os aplicativos podem registrar listeners (ouvintes) para acompanhar alterações nos dados.
- Quando ocorre uma alteração, o Firestore notifica os clientes interessados, permitindo que a interface seja atualizada automaticamente.
- Os dados são organizados em uma estrutura semelhante a uma árvore JSON.

Firestore Realtime Database

- É adequado para aplicações que precisam de atualizações frequentes e em tempo real, como:
 - Chats e mensagens;
 - Aplicativos colaborativos;
 - Sistemas de acompanhamento em tempo real;
 - Jogos e aplicações interativas.
- Foi um dos primeiros serviços de banco de dados disponibilizados pelo Firebase.

Firestore Realtime Database x SQLite/Room

Firestore Real Time Database

Banco de dados na nuvem

Dados armazenados na infraestrutura do
Firestore

Permite acesso e sincronização entre
dispositivos

Sincronização em tempo real

Ideal para aplicações conectadas e
colaborativas

SQLite/Room

Banco de dados Local

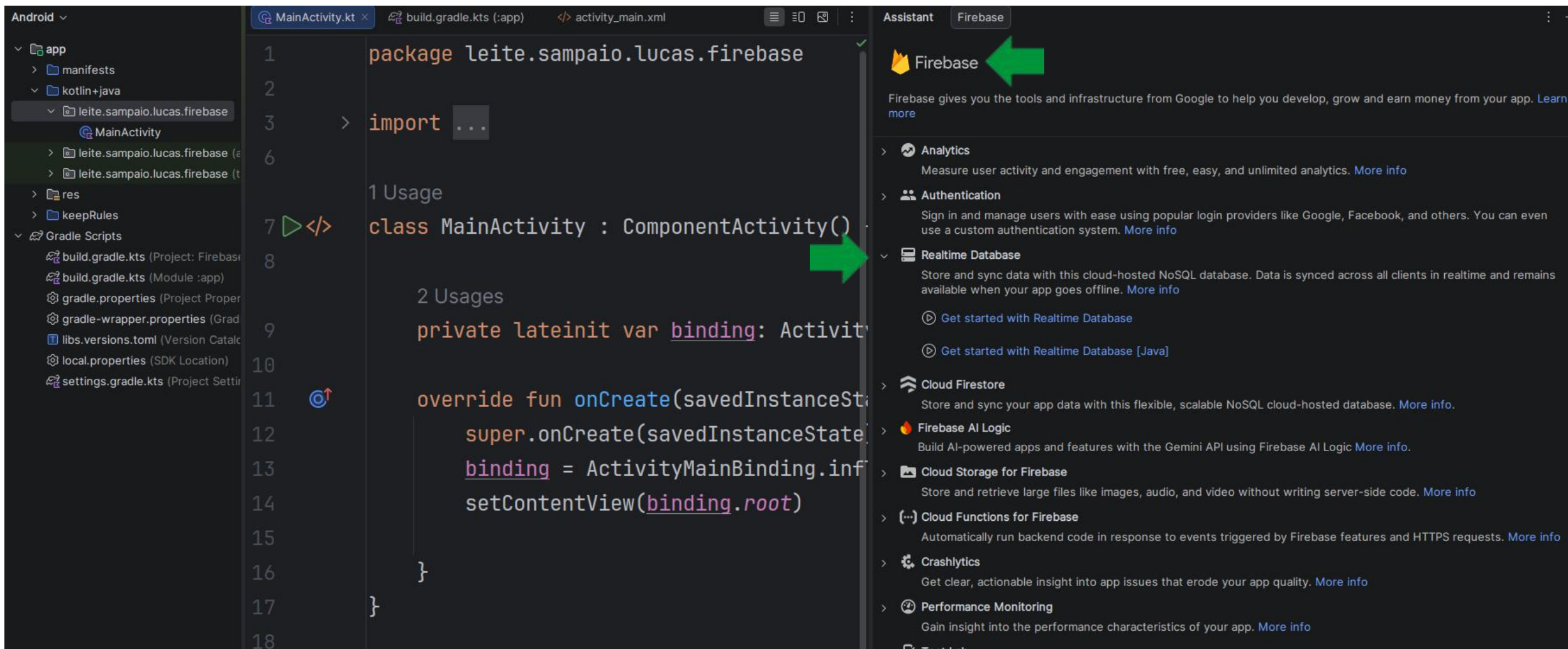
Dados armazenados no dispositivo

Acesso principalmente offline

Não possui sincronização automática entre
usuários

Ideal para persistência local

Integração com o Firebase Realtime Database



The screenshot displays the Android Studio IDE interface for integrating Firebase Realtime Database into a Kotlin project.

Left Panel (Project Structure): The project structure shows the following hierarchy:

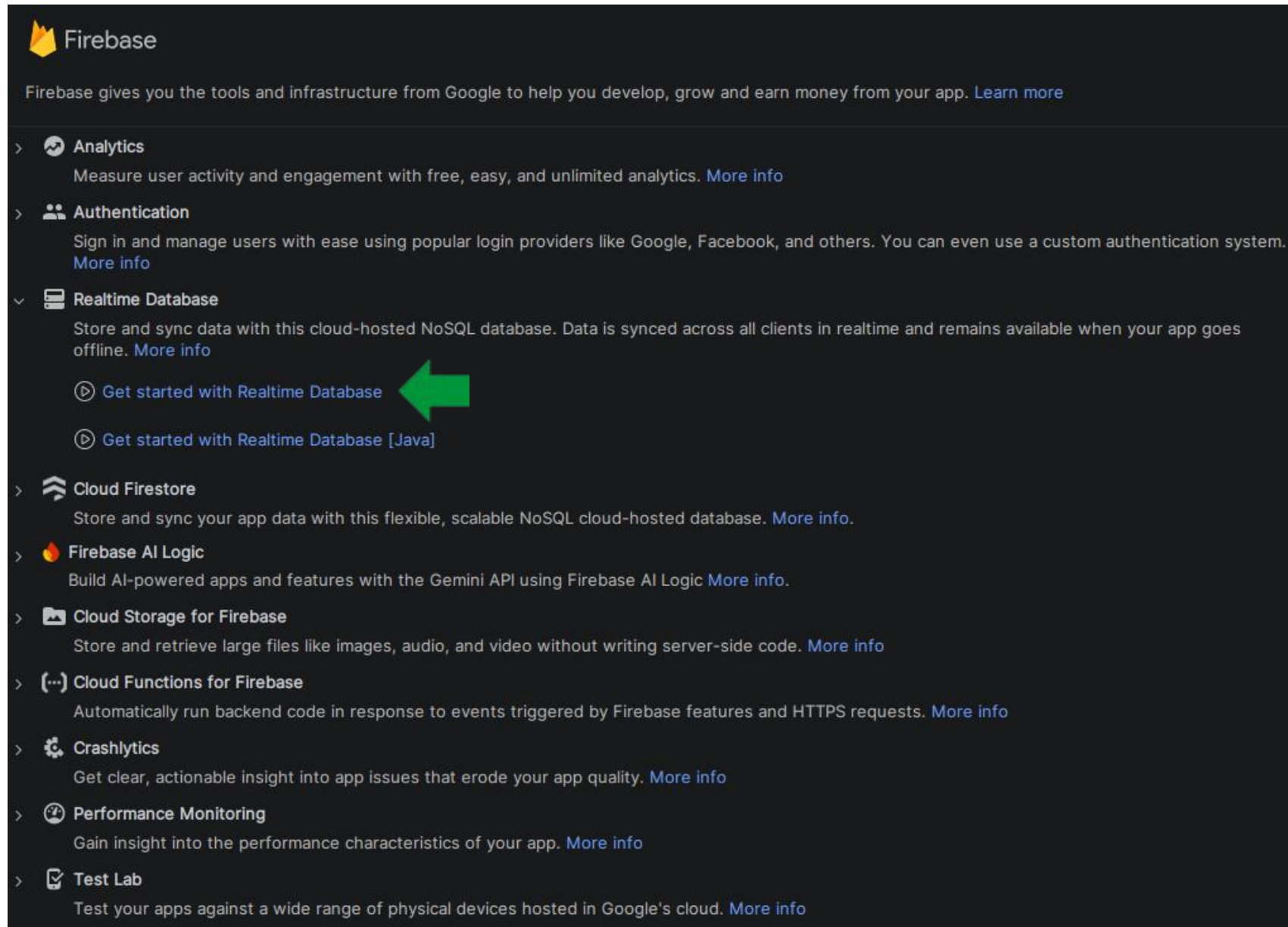
- app
 - manifests
 - kotlin+java
 - leite.sampaio.lucas.firebaseio
 - MainActivity
 - res
 - keepRules
- Gradle Scripts
 - build.gradle.kts (Project: Firebase)
 - build.gradle.kts (Module :app)
 - gradle.properties (Project Properties)
 - gradle-wrapper.properties (Gradle Wrapper)
 - libs.versions.toml (Version Catalog)
 - local.properties (SDK Location)
 - settings.gradle.kts (Project Settings)

Main Editor (MainActivity.kt): The code in MainActivity.kt is as follows:

```
1 package leite.sampaio.lucas.firebaseio
2
3 > import ...
4
5 1 Usage
6
7 class MainActivity : AppCompatActivity() {
8
9     2 Usages
10    private lateinit var binding: ActivityMainBinding
11
12    override fun onCreate(savedInstanceState: Bundle?) {
13        super.onCreate(savedInstanceState)
14        binding = ActivityMainBinding.inflate(layoutInflater)
15        setContentView(binding.root)
16    }
17 }
18
```

Right Panel (Assistant): The Assistant panel shows the Firebase integration options. The **Realtime Database** option is selected, indicated by a green arrow. The description for Realtime Database is: "Store and sync data with this cloud-hosted NoSQL database. Data is synced across all clients in realtime and remains available when your app goes offline. [More info](#)".

Integração com o Firebase Realtime Database

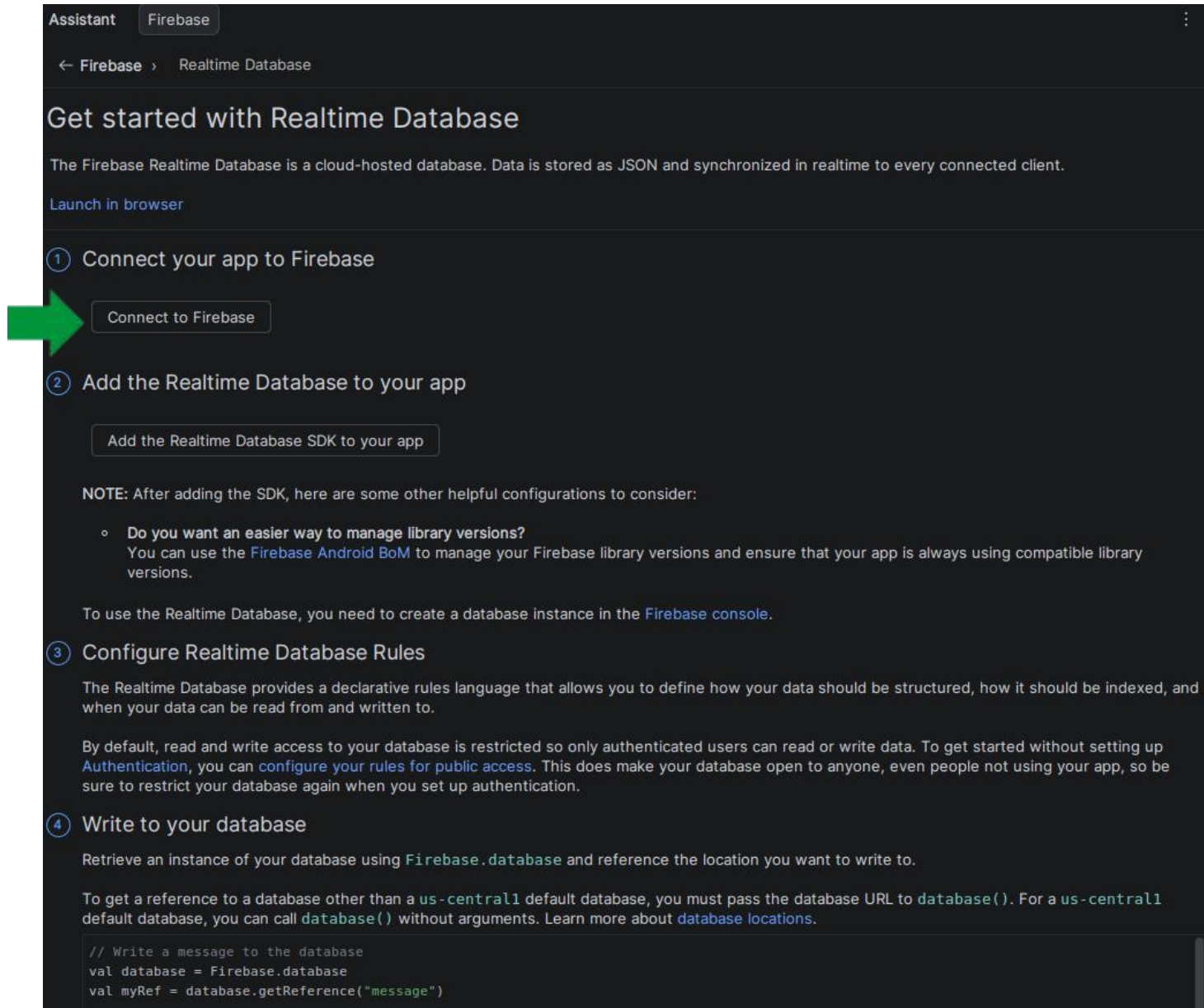


Firebase

Firebase gives you the tools and infrastructure from Google to help you develop, grow and earn money from your app. [Learn more](#)

- > **Analytics**
Measure user activity and engagement with free, easy, and unlimited analytics. [More info](#)
- > **Authentication**
Sign in and manage users with ease using popular login providers like Google, Facebook, and others. You can even use a custom authentication system. [More info](#)
- ✓ **Realtime Database**
Store and sync data with this cloud-hosted NoSQL database. Data is synced across all clients in realtime and remains available when your app goes offline. [More info](#)
 - ▶ [Get started with Realtime Database](#) ←
 - ▶ [Get started with Realtime Database \[Java\]](#)
- > **Cloud Firestore**
Store and sync your app data with this flexible, scalable NoSQL cloud-hosted database. [More info](#).
- > **Firebase AI Logic**
Build AI-powered apps and features with the Gemini API using Firebase AI Logic [More info](#).
- > **Cloud Storage for Firebase**
Store and retrieve large files like images, audio, and video without writing server-side code. [More info](#)
- > **Cloud Functions for Firebase**
Automatically run backend code in response to events triggered by Firebase features and HTTPS requests. [More info](#)
- > **Crashlytics**
Get clear, actionable insight into app issues that erode your app quality. [More info](#)
- > **Performance Monitoring**
Gain insight into the performance characteristics of your app. [More info](#)
- > **Test Lab**
Test your apps against a wide range of physical devices hosted in Google's cloud. [More info](#)

Integração com o Firebase Realtime Database



Assistant **Firebase**

← Firebase > Realtime Database

Get started with Realtime Database

The Firebase Realtime Database is a cloud-hosted database. Data is stored as JSON and synchronized in realtime to every connected client.

[Launch in browser](#)

- 1 Connect your app to Firebase
 [Connect to Firebase](#)
- 2 Add the Realtime Database to your app
[Add the Realtime Database SDK to your app](#)

NOTE: After adding the SDK, here are some other helpful configurations to consider:

- Do you want an easier way to manage library versions?
You can use the [Firebase Android BoM](#) to manage your Firebase library versions and ensure that your app is always using compatible library versions.

To use the Realtime Database, you need to create a database instance in the [Firebase console](#).

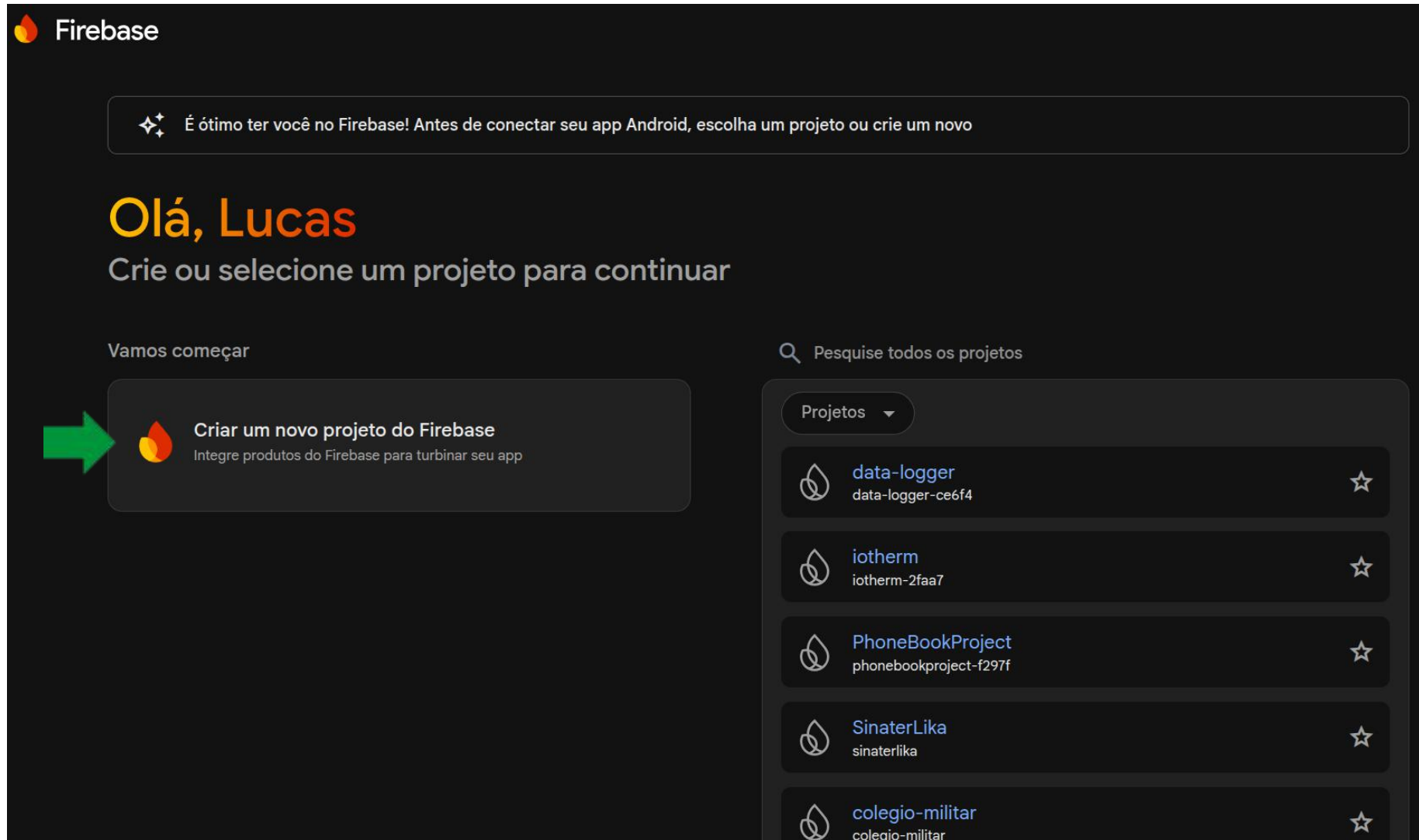
- 3 Configure Realtime Database Rules
The Realtime Database provides a declarative rules language that allows you to define how your data should be structured, how it should be indexed, and when your data can be read from and written to.

By default, read and write access to your database is restricted so only authenticated users can read or write data. To get started without setting up [Authentication](#), you can [configure your rules for public access](#). This does make your database open to anyone, even people not using your app, so be sure to restrict your database again when you set up authentication.
- 4 Write to your database
Retrieve an instance of your database using `Firebase.database` and reference the location you want to write to.

To get a reference to a database other than a `us-central1` default database, you must pass the database URL to `database()`. For a `us-central1` default database, you can call `database()` without arguments. Learn more about [database locations](#).

```
// Write a message to the database
val database = Firebase.database
val myRef = database.getReference("message")
```

Integração com o Firebase Realtime Database



Integração com o Firebase Realtime Database

× Criar um projeto

Vamos começar nomeando o
projeto[?]

Nome do projeto

firebase-android ←

 fir-android-91964

Já tem um projeto do Google Cloud?

[Adicionar o Firebase ao projeto do Google Cloud](#)

Continuar ←



Integração com o Firebase Realtime Database

× Criar um projeto

Assistência de IA para seu projeto do Firebase

O Gemini está integrado ao console do Firebase para ajudar a simplificar o processo de desenvolvimento.

- ✦ Converse com o Gemini para planejar e criar seu aplicativo, solucionar problemas e receber recomendações personalizadas para seu projeto
- ⚙️ Depure e resolva problemas nos seus apps usando a assistência de IA no Firebase Crashlytics
- 📩 Receba insights e recomendações sobre campanhas com o Firebase Cloud Messaging para se conectar melhor com os usuários
- 🔗 Gere esquemas e analise seus dados usando linguagem natural no Firebase SQL Connect

☒ Ativar o Gemini no Firebase
Recomendado

Exonerações de responsabilidade:

O Gemini no Firebase ainda está em desenvolvimento e pode fornecer informações incorretas. Teste os códigos gerados antes de usá-los nos seus apps. O uso do Gemini no Firebase (sem custo financeiro) é regido pelos [Termos de Serviço do Google](#) e pela [Política de Uso Proibido da IA Generativa](#). Por isso, o Gemini pode usar seus comandos e as respectivas respostas para treinar o modelo.

[Anterior](#)

[Continuar](#)



Integração com o Firebase Realtime Database

× Criar um projeto

Google Analytics para seu projeto do Firebase

O Google Analytics é uma solução de análise ilimitada e gratuita. Com ele, é possível segmentar, gerar relatórios e muito mais nos seguintes produtos: Firebase Crashlytics, Cloud Messaging, Mensagens no app, Configuração remota, Teste A/B e Cloud Functions.

O Google Analytics ativa:

- × Teste A/B ?
- × Segmentação de usuários em produtos do Firebase ?
- × Registros de navegação estrutural no Crashlytics ?
- × Gatilhos do Cloud Functions com base em eventos ?
- × Geração de relatórios ilimitada gratuita ?

☒ Ativar o Google Analytics neste projeto
Recomendado

Anterior

Criar projeto



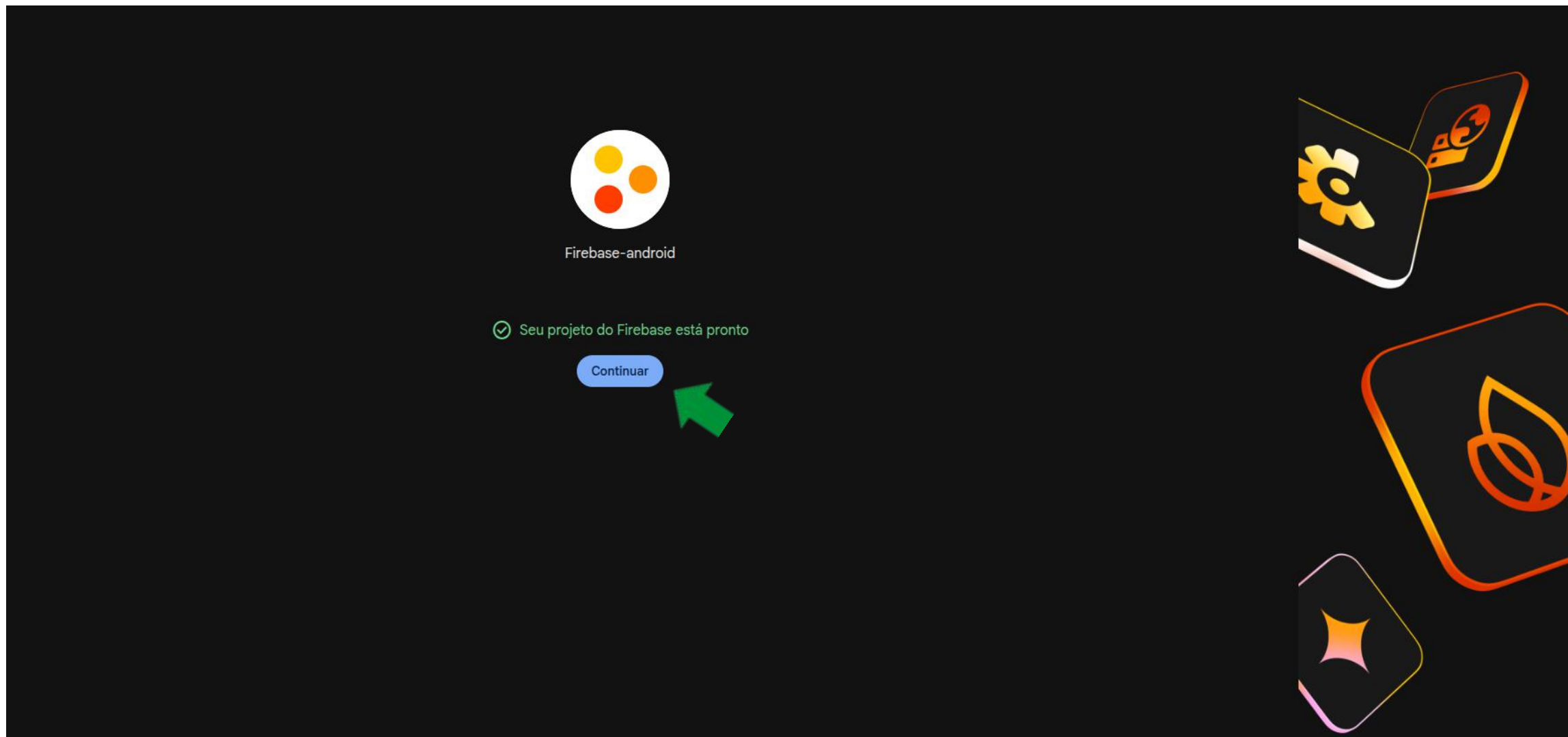
Integração com o Firebase Realtime Database



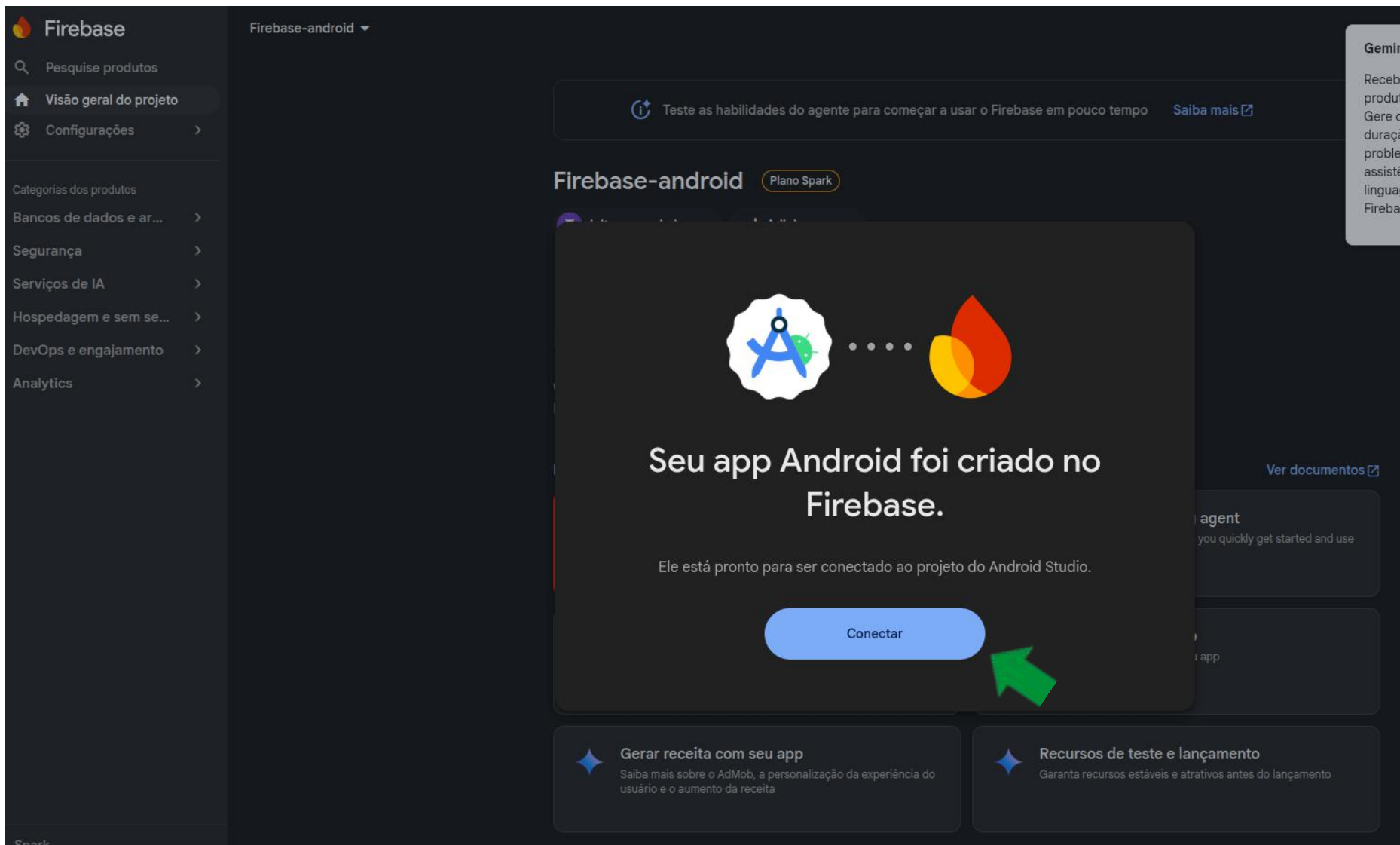
Preparando seu projeto, aguarde.
Firebase-android



Integração com o Firebase Realtime Database



Integração com o Firebase Realtime Database



The screenshot shows the Firebase console interface. On the left is a sidebar with the 'Firebase' logo and a search bar. Below the search bar are navigation links: 'Visão geral do projeto' (selected), 'Configurações', 'Categorias dos produtos', 'Bancos de dados e ar...', 'Segurança', 'Serviços de IA', 'Hospedagem e sem se...', 'DevOps e engajamento', and 'Analytics'. The main area displays the 'Firebase-android' project page. At the top, there's a banner with a clock icon and the text 'Teste as habilidades do agente para começar a usar o Firebase em pouco tempo' and a 'Saiba mais' link. Below this, the 'Firebase-android' title is followed by a 'Plano Spark' badge. A large modal dialog is centered on the screen, featuring the Android Studio logo and the Firebase logo connected by a dotted line. The dialog text reads: 'Seu app Android foi criado no Firebase.' and 'Ele está pronto para ser conectado ao projeto do Android Studio.' Below the text is a blue 'Conectar' button, which is highlighted by a green arrow. At the bottom of the modal, there are two sections: 'Gerar receita com seu app' with a subtext 'Saiba mais sobre o AdMob, a personalização da experiência do usuário e o aumento da receita' and 'Recursos de teste e lançamento' with a subtext 'Garanta recursos estáveis e atrativos antes do lançamento'. On the right side of the console, there's a 'Gemin' sidebar with text: 'Receba produtos', 'Gere co', 'duraça', 'proble', 'assistê', 'linguag', and 'Firebas'. Below this is a 'Ver documentos' link and a section titled 'agent' with the text 'you quickly get started and use'.

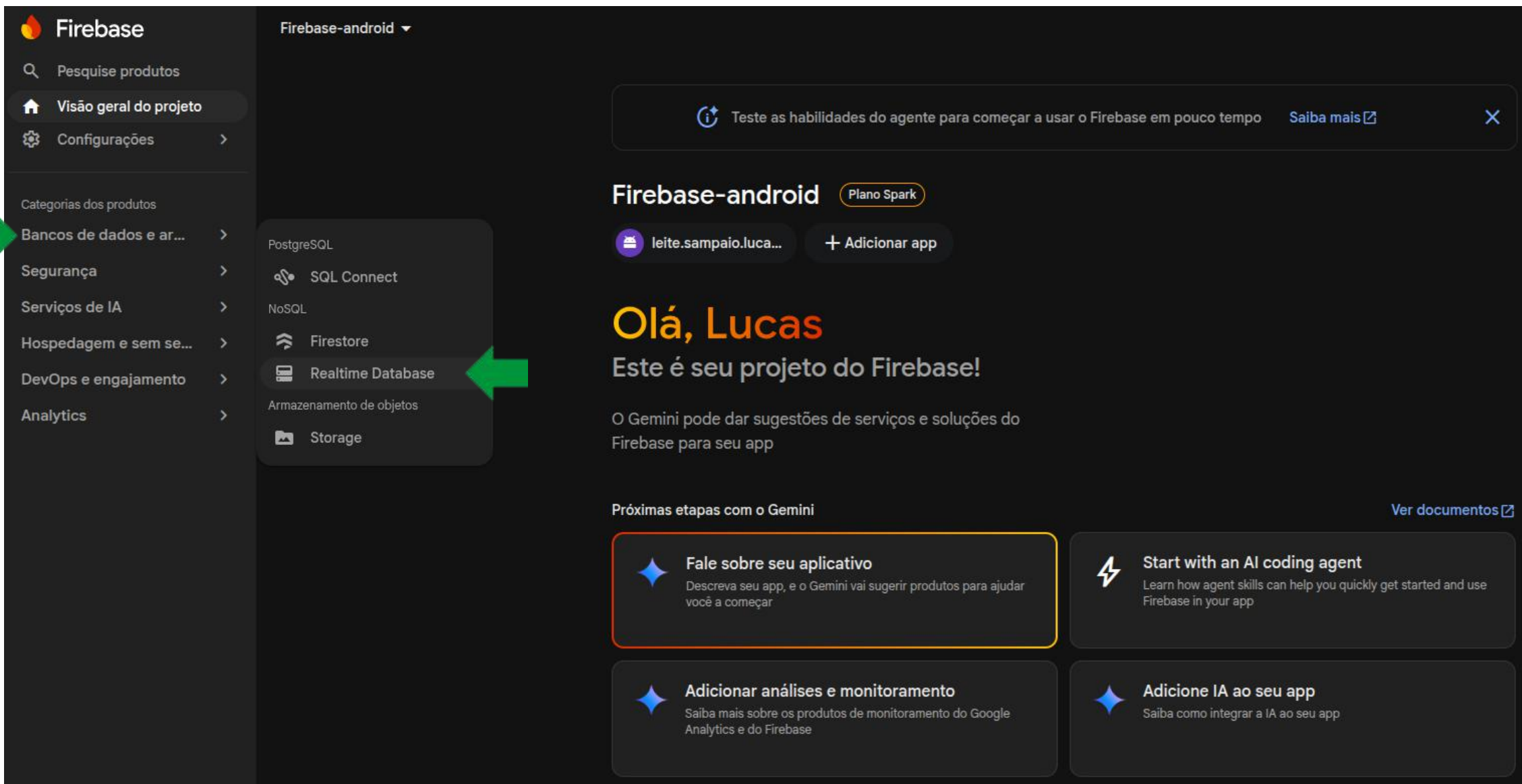
Integração com o Firebase Realtime Database



**Your Android Studio project is connected
to your Firebase Android app**

You can now use Firebase in your project! Go back to Android Studio to start using one of the
Firebase SDKs.

Integração com o Firebase Realtime Database



Firestore

Pesquise produtos

Visão geral do projeto

Configurações

Categorias dos produtos

Bancos de dados e ar... >

Segurança >

Serviços de IA >

Hospedagem e sem se... >

DevOps e engajamento >

Analytics >

PostgreSQL

SQL Connect

NoSQL

Firestore

Realtime Database

Armazenamento de objetos

Storage

Teste as habilidades do agente para começar a usar o Firebase em pouco tempo [Saiba mais](#)

Firebase-android **Plano Spark**

leite.sampaio.luca... + Adicionar app

Olá, Lucas

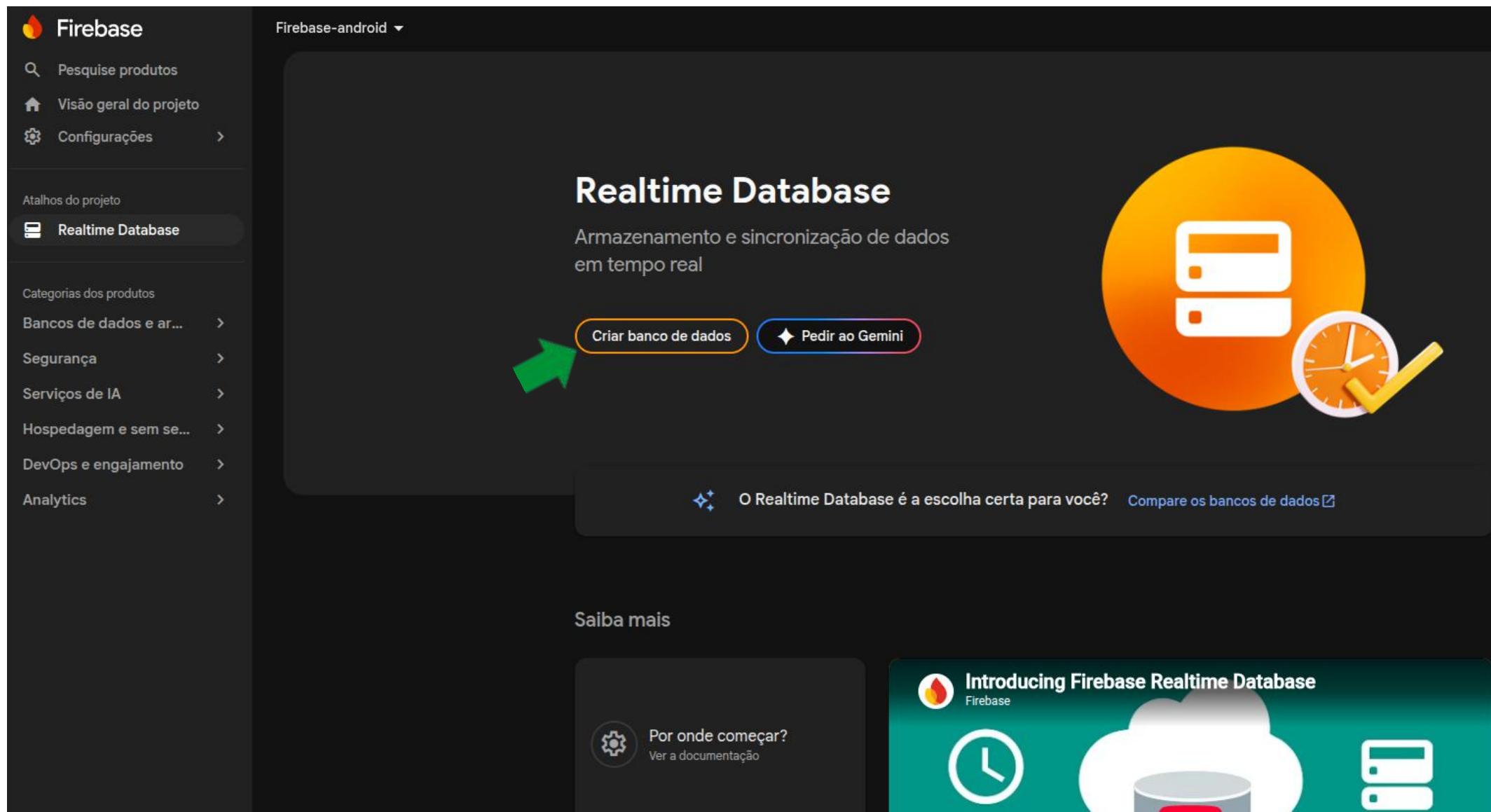
Este é seu projeto do Firebase!

O Gemini pode dar sugestões de serviços e soluções do Firebase para seu app

Próximas etapas com o Gemini [Ver documentos](#)

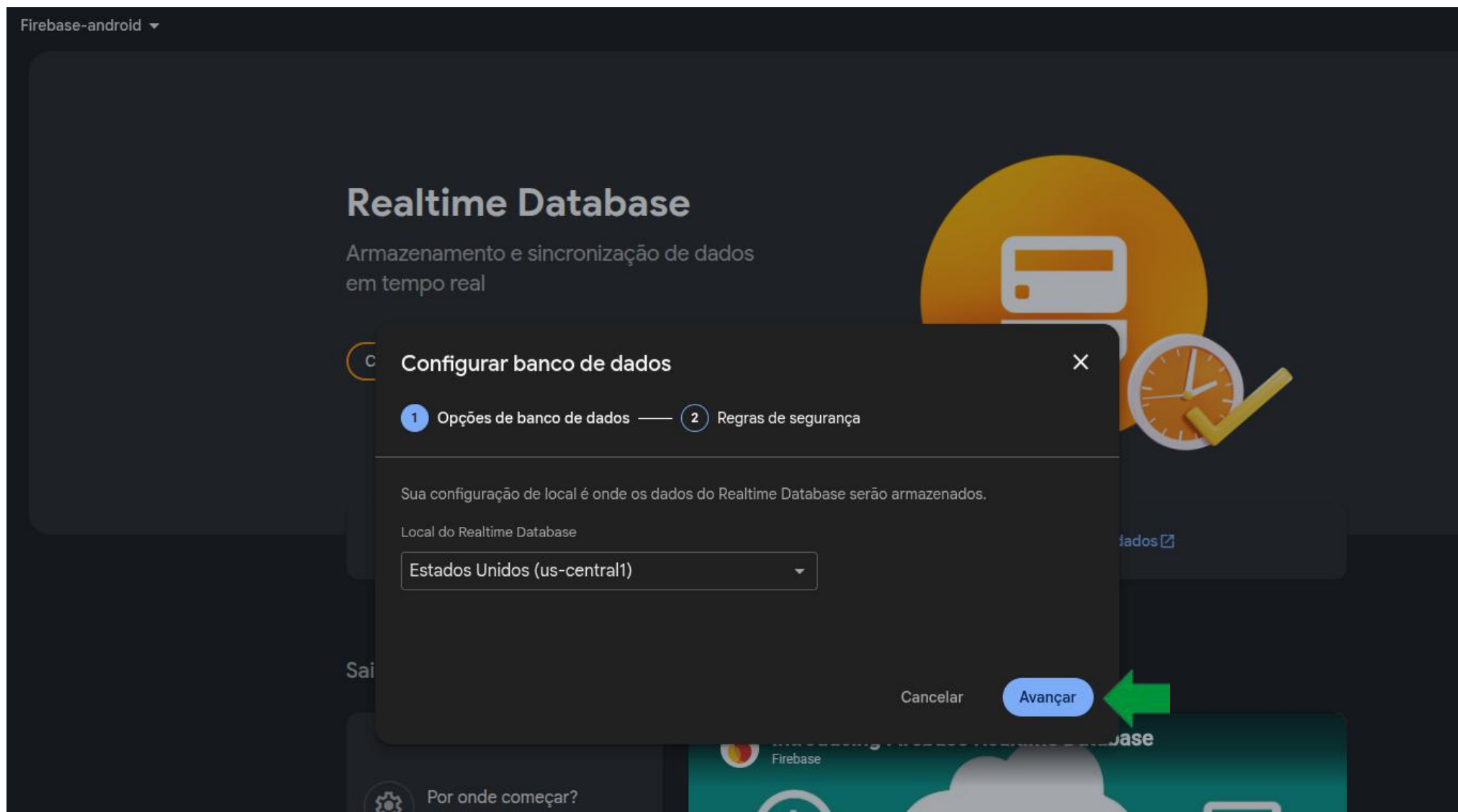
- Fale sobre seu aplicativo**
Descreva seu app, e o Gemini vai sugerir produtos para ajudar você a começar
- Start with an AI coding agent**
Learn how agent skills can help you quickly get started and use Firebase in your app
- Adicionar análises e monitoramento**
Saiba mais sobre os produtos de monitoramento do Google Analytics e do Firebase
- Adicione IA ao seu app**
Saiba como integrar a IA ao seu app

Integração com o Firebase Realtime Database



The screenshot displays the Firebase console interface. On the left, a sidebar contains the 'Firebase' logo and navigation links: 'Pesquise produtos', 'Visão geral do projeto', 'Configurações', 'Atalhos do projeto', 'Realtime Database' (highlighted), 'Bancos de dados e ar...', 'Segurança', 'Serviços de IA', 'Hospedagem e sem se...', 'DevOps e engajamento', and 'Analytics'. The main content area is titled 'Realtime Database' and includes the subtitle 'Armazenamento e sincronização de dados em tempo real'. It features two buttons: 'Criar banco de dados' (highlighted with a green arrow) and 'Pedir ao Gemini'. Below these buttons is a banner that reads 'O Realtime Database é a escolha certa para você?' with a link to 'Compare os bancos de dados'. At the bottom, there is a 'Saiba mais' section with a link 'Por onde começar? Ver a documentação' and a video thumbnail titled 'Introducing Firebase Realtime Database'.

Integração com o Firebase Realtime Database



Integração com o Firebase Realtime Database

Configurar banco de dados

Opções de banco de dados

2 Regras de segurança

Após definir a estrutura de dados, escreva regras para proteger os dados.
[Saiba mais](#)

☐ Iniciar no modo bloqueado
Seus dados são privados por padrão. O acesso de leitura/gravação do cliente será concedido apenas se especificado por suas regras de segurança.

☒ Iniciar no modo de teste
Por padrão, seus dados estão definidos para permitir uma configuração rápida. Porém, você precisa atualizar suas regras de segurança em até 30 dias para permitir em longo prazo o acesso de leitura/gravação do cliente.

```
{
  "rules": {
    ".read": "now < 1792983600000", // 2026-10-26
    ".write": "now < 1792983600000", // 2026-10-26
  }
}
```

 As regras de segurança padrão para o modo de teste permitem que qualquer pessoa com a referência do seu banco de dados acesse, edite e exclua todos os dados nos próximos 30 dias

Cancelar

 Ativar

Integração com o Firebase Realtime Database

Firestore-android ▾

Realtime Database

◆ Precisa de ajuda com o Realtime Database? Peça ao Gemini!

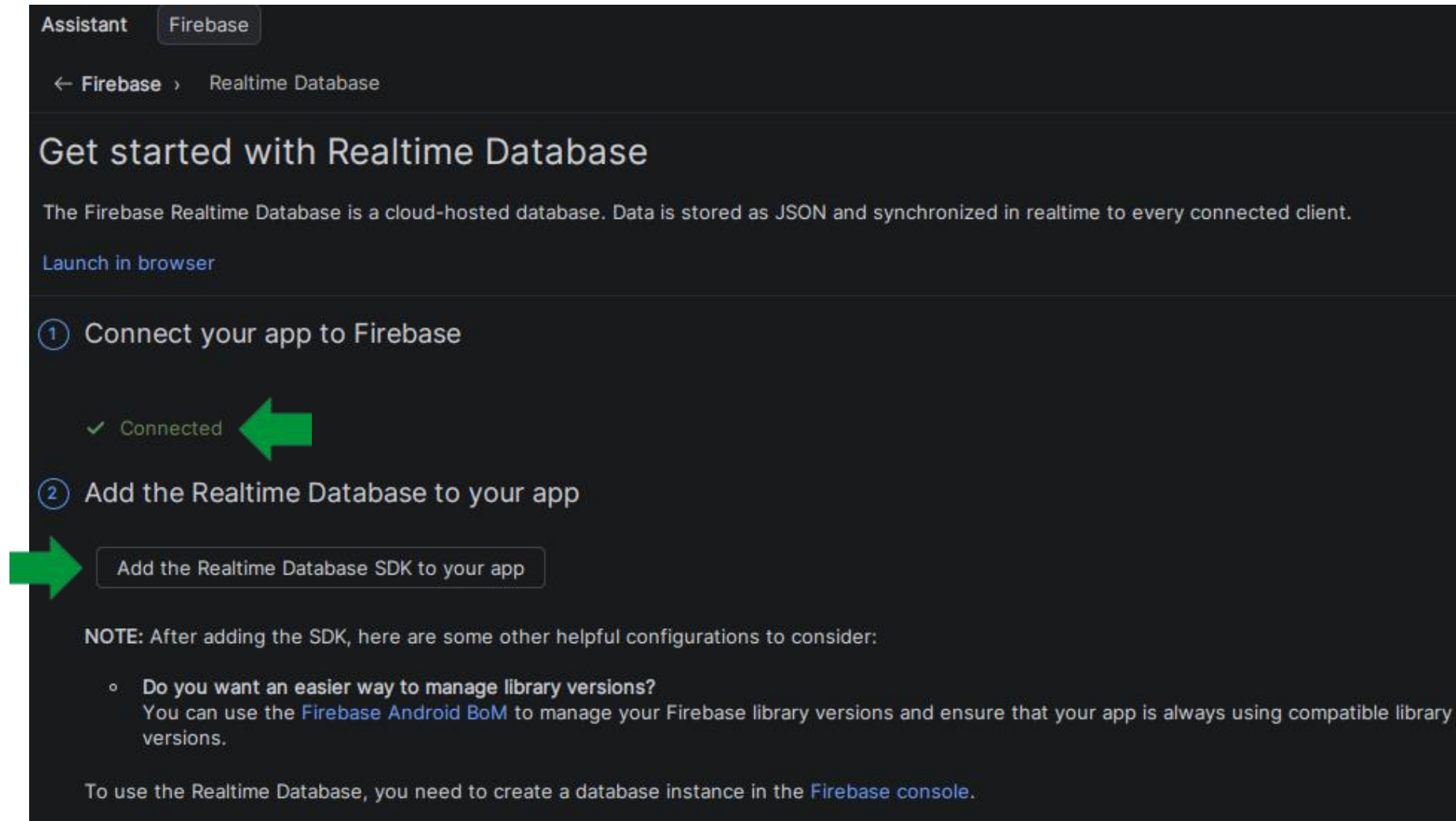
Dados Regras Backups Uso

Proteja os recursos do Realtime Database de abusos, como fraude de faturamento ou phishing [Configurar o App Check](#) ✕

<https://fir-android-91964-default-rtdb.firebaseio.com>

`https://fir-android-91964-default-rtdb.firebaseio.com/: null`

Integração com o Firebase Realtime Database



Assistant **Firebase**

← Firebase > Realtime Database

Get started with Realtime Database

The Firebase Realtime Database is a cloud-hosted database. Data is stored as JSON and synchronized in realtime to every connected client.

[Launch in browser](#)

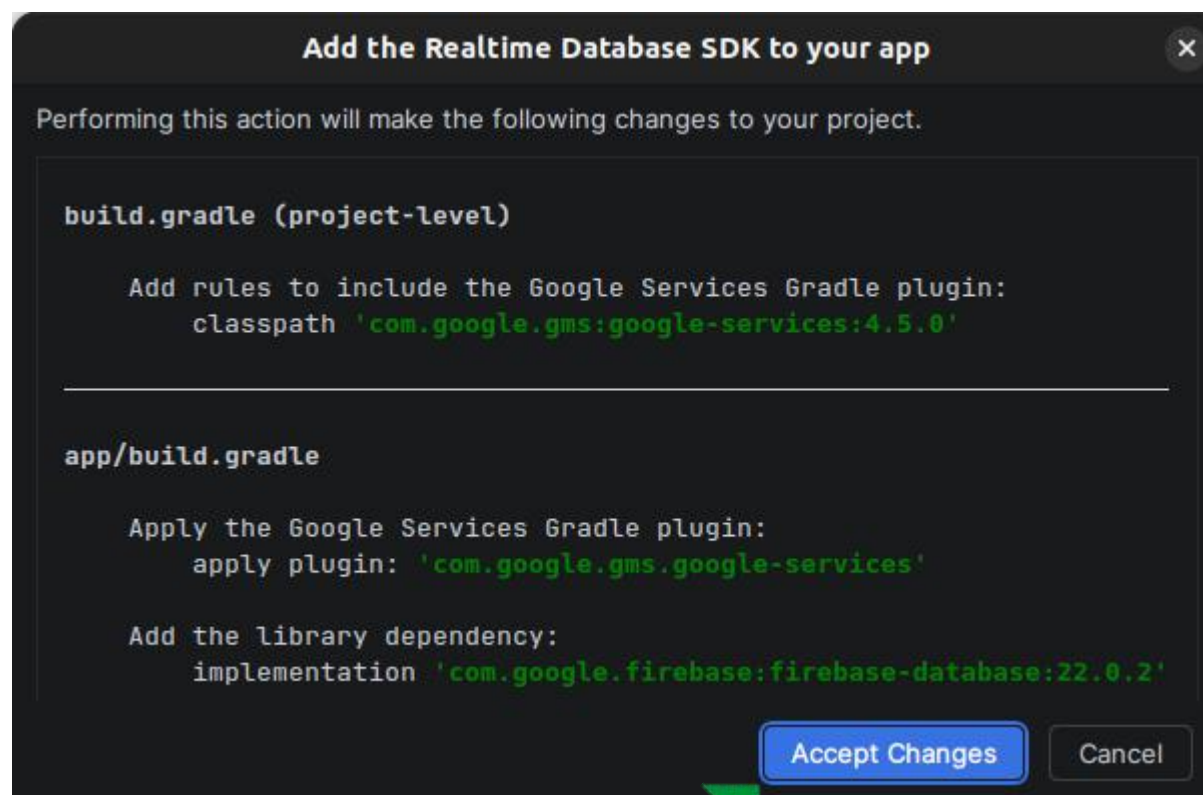
- 1 Connect your app to Firebase
 - ✓ Connected
- 2 Add the Realtime Database to your app
 - [Add the Realtime Database SDK to your app](#)

NOTE: After adding the SDK, here are some other helpful configurations to consider:

- Do you want an easier way to manage library versions?
You can use the [Firebase Android BoM](#) to manage your Firebase library versions and ensure that your app is always using compatible library versions.

To use the Realtime Database, you need to create a database instance in the [Firebase console](#).

Integração com o Firebase Realtime Database



Integração com o Firebase Realtime Database

- build.gradle.kts (Project:RoomSQLite)

```
plugins {  
    alias(libs.plugins.google.gms.google.services) apply false
```

- build.gradle.kts (Module:app)

```
plugins {  
    alias(libs.plugins.google.gms.google.services)
```

```
dependencies {  
  
    implementation(libs.firebase.database)
```

Testando a integração com o Firebase Realtime Database

```
class MainActivity : ComponentActivity() {  
  
    2 Usages  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        ➡ val db = Firebase.database  
        val myRef = db.getReference(path = "message")  
  
        myRef.setValue("Hello World!") ←  
    }  
}
```

Testando a integração com o Firebase Realtime Database

Firestore-android ▾

Realtime Database

◆ Precisa de ajuda com o Realtime Database? Peça ao Gemini!

Dados Regras Backups Uso

Proteja os recursos do Realtime Database de abusos, como fraude de faturamento ou phishing [Configurar o App Check](#) ✕

```
https://fir-android-91964-default-rtdb.firebaseio.com/

└─ message: "Hello World!"
```



Implementando o BookRepository

```
class BookRepository {  
  
    6 Usages  
    private val booksReference: DatabaseReference =  
        FirebaseDatabase  
            .getInstance()  
            .reference  
            .child(pathString = "books")  
}
```

Implementando o BookRepository

```
fun observeBooks(listener: ValueEventListener) {  
    booksReference.addValueEventListener(listener)  
}
```

Registra um ValueEventListener na booksReference para receber os dados e ser notificada sempre que eles forem alterados no Firebase Realtime Database.

1 Usage

```
fun removeObserver(listener: ValueEventListener) {  
    booksReference.removeEventListener(listener)  
}
```

Remove o ValueEventListener previamente registrado na booksReference, interrompendo o recebimento de atualizações do Firebase.

Implementando o BookRepository

```
fun insert(book: Book, onComplete: (Boolean, String?) -> Unit) {  
    val key = booksReference.push().key  
  
    if (key == null) {  
        onComplete(false, "Não foi possível gerar a chave do livro.")  
        return  
    }  
  
    val newBook = book.copy(id = key)  
  
    booksReference  
        .child(pathString = key)  
        .setValue(newBook)  
        .addOnCompleteListener { task ->  
            onComplete(  
                task.isSuccessful,  
                task.exception?.message  
            )  
        }  
}
```

No Realtime Database, os IDs gerados por push() são chaves únicas chamadas Push IDs, que são únicas e ordenáveis cronologicamente.

Inicia uma operação assíncrona do Firebase para inserir ou atualizar o dado no servidor e retorna imediatamente um Task.

Implementando o BookRepository

```
fun update(book: Book, onComplete: (Boolean, String?) -> Unit) {  
    booksReference  
        .child(pathString = book.id)  
        .setValue(book)  
        .addOnCompleteListener { task ->  
            onComplete(  
                task.isSuccessful,  
                task.exception?.message  
            )  
        }  
}
```

Implementando o BookRepository

```
fun delete(book: Book, onComplete: (Boolean, String?) -> Unit) {  
    booksReference  
        .child(pathString = book.id)  
        .removeValue()  
        .addOnCompleteListener { task ->  
            onComplete(  
                task.isSuccessful,  
                task.exception?.message  
            )  
        }  
}
```

Implementando a MainActivity

```
private val booksListener = object : ValueEventListener {

    override fun onDataChange(snapshot: DataSnapshot) {
        val books = snapshot.children.mapNotNull {
            it.getValue(valueType = Book::class.java)
        }.sortedBy { it.title.lowercase() }

        adapter.updateList(novosLivros = books)
    }

    override fun onCancelled(error: DatabaseError) {
        Toast.makeText(
            context = this@MainActivity,
            text = "Erro ao carregar livros: ${error.message}",
            duration = Toast.LENGTH_LONG
        ).show()
    }
}
```

Implementando a MainActivity

```
private val booksListener = object : ValueEventListener {  
  
    override fun onDataChange(snapshot: DataSnapshot) {  
        val books = snapshot.children.mapNotNull {  
            it.getValue(valueType = Book::class.java)  
        }.sortedBy { it.title.lowercase() }  
  
        adapter.updateList(novosLivros = books)  
    }  
  
    override fun onCancelled(error: DatabaseError) {  
        Toast.makeText(  
            context = this@MainActivity,  
            text = "Erro ao carregar livros: ${error.message}",  
            duration = Toast.LENGTH_LONG  
        ).show()  
    }  
}
```

onDataChange: É chamado quando os dados do Firebase são carregados ou alterados, permitindo atualizar a interface com os dados mais recentes.

onCancelled: É chamado quando ocorre um erro ou o acesso aos dados é cancelado, permitindo tratar a falha.

Implementando a MainActivity

```
private lateinit var repository: BookRepository
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    ➡ repository = BookRepository()  
  
    configureRecyclerView()  
    configureButtons()  
    observeBooks()  
}
```

Implementando a MainActivity

```
private fun configureRecyclerView() {  
    adapter = BookAdapter(  
        onItemClick = { book -> selectBook(book) },  
        onDeleteClick = { book -> deleteBook(book) }  
    )  
  
    binding.rvBooks.layoutManager = LinearLayoutManager(context = this)  
    binding.rvBooks.adapter = adapter  
}
```

Implementando a MainActivity

```
private fun configureButtons() {  
    binding.btnSave.setOnClickListener {  
        insertBook()  
    }  
  
    binding.btnUpdate.setOnClickListener {  
        updateBook()  
    }  
}
```

Implementando a MainActivity

```
private fun insertBook() {  
    val title = binding.etTitle.text.toString().trim()  
    val author = binding.etAuthor.text.toString().trim()  
    val yearText = binding.etYear.text.toString().trim()  
  
    if (title.isBlank() || author.isBlank() || yearText.isBlank()) {  
        Toast.makeText(context = this, text = "Preencha todos os campos.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
  
    val year = yearText.toIntOrNull()  
  
    if (year == null) {  
        Toast.makeText(context = this, text = "Informe um ano válido.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
}
```



Implementando a MainActivity



```
val book = Book(  
    title = title,  
    author = author,  
    year = year  
)  
  
repository.insert(book) { success, error ->  
    if (success) {  
        clearForm()  
        Toast.makeText(context = this, text = "Livro inserido.", duration = Toast.LENGTH_SHORT).show()  
    } else {  
        Toast.makeText(  
            context = this,  
            text = "Erro ao inserir livro: ${error ?: "não foi possível gerar a chave."}",  
            duration = Toast.LENGTH_LONG  
        ).show()  
    }  
}
```

Implementando a MainActivity

```
private fun updateBook() {  
    val book = selectedBook  
  
    if (book == null) {  
        Toast.makeText(context = this, text = "Selecione um livro.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
  
    val title = binding.etTitle.text.toString().trim()  
    val author = binding.etAuthor.text.toString().trim()  
    val yearText = binding.etYear.text.toString().trim()  
  
    if (title.isBlank() || author.isBlank() || yearText.isBlank()) {  
        Toast.makeText(context = this, text = "Preencha todos os campos.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
}
```



Implementando a MainActivity



```
val year = yearText.toIntOrNull()

if (year == null) {
    Toast.makeText(context = this, text = "Informe um ano válido.", duration = Toast.LENGTH_SHORT).show()
    return
}

val updatedBook = book.copy(title = title, author = author, year = year)

repository.update(updatedBook) { success, error ->
    if (success) {
        clearForm()
        Toast.makeText(context = this, text = "Livro atualizado.", duration = Toast.LENGTH_SHORT).show()
    } else {
        Toast.makeText(context = this, text = "Erro ao atualizar livro: ${error ?: "erro desconhecido."}",
            duration = Toast.LENGTH_LONG).show()
    }
}
}
```

Implementando a MainActivity

```
private fun deleteBook(book: Book) {  
    repository.delete(book) { success, error ->  
        if (success) {  
            if (selectedBook?.id == book.id) {  
                clearForm()  
            }  
  
            Toast.makeText(context = this, text = "Livro excluído.", duration = Toast.LENGTH_SHORT).show()  
        } else {  
            Toast.makeText(  
                context = this,  
                text = "Erro ao excluir livro: ${error ?: "erro desconhecido."}",  
                duration = Toast.LENGTH_LONG  
            ).show()  
        }  
    }  
}
```

Implementando a MainActivity

```
private fun selectBook(book: Book) {  
    selectedBook = book  
  
    binding.etTitle.setText(book.title)  
    binding.etAuthor.setText(book.author)  
    binding.etYear.setText(book.year.toString())  
  
    Toast.makeText(context = this, text = "Livro selecionado.", duration = Toast.LENGTH_SHORT).show()  
}
```

Implementando a MainActivity

```
private fun clearForm() {  
    binding.etTitle.text.clear()  
    binding.etAuthor.text.clear()  
    binding.etYear.text.clear()  
    selectedBook = null  
}  
  
1 Usage  
private fun observeBooks() {  
    repository.observeBooks(booksListener)  
}
```

Implementando a MainActivity

```
override fun onDestroy() {  
    repository.removeObserver(booksListener)  
    super.onDestroy()  
}
```

Exercícios

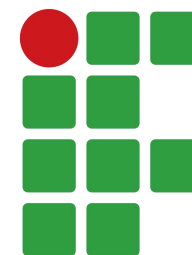
- Modifique o projeto de gerenciamento de livros desenvolvido anteriormente com Room/SQLite para utilizar o Firebase Realtime Database.
 - Faça a integração do projeto com o Firebase.
 - Reaproveite o BookAdapter utilizado no projeto anterior.
 - Modifique a classe Book, removendo as anotações do Room.
 - Substitua o BookDao por um BookRepository, responsável pelas operações no Firebase.
 - Modifique a MainActivity para utilizar o BookRepository.
 - Mantenha as operações de cadastrar, listar, atualizar e excluir livros.

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco